



Department of Computer Engineering

Faculty of Engineering

University of Ruhuna, Sri Lanka

EC8207: Applied Big Data Engineering

Scenario 2: FinTech Fraud Detection Pipeline

Domain: Banking & Security

EG/2020/4088 – Navoda K.D.L.

EG/2020/4096 – Nethsarani D.W.D.

Contents

1. Introduction	1
2. System Architecture	1
3. Justification of Tools Selected.....	2
3.1 Apache Kafka	2
3.2 Apache Spark Structured Streaming.....	3
3.3 Apache Airflow.....	3
4. Event Time vs. Processing Time	3
5. Pipeline Output & Results.....	4
5.1 Producer Output	4
5.2 Spark Streaming Output	6
5.3 Airflow DAG.....	6
5.4 Reconciliation Report.....	7
5.5 Analytic Report – Fraud Attempts by Merchant Category	8
6. Ethics and Data Governance	10
6.1 Privacy Implications.....	10
6.2 Data Governance Recommendations	10
7. Conclusion.....	11

List of Figures

Figure 1: End-to-end Lambda architecture of the FinTech Fraud Detection Pipeline	1
Figure 2: Python producer generating transaction stream with occasional fraud injections. 5	
Figure 3: Spark Structured Streaming detecting fraud in real-time micro-batches.....	6
Figure 4: Apache Airflow DAG successfully completing ETL and reconciliation tasks.....	7
Figure 5: Reconciliation report comparing Total Ingress Amount vs. Validated Amount. ...	8
Figure 6: Fraud Attempts by Merchant Category — the key analytic deliverable.....	9

List of Tables

Table 1: End-to-end Lambda architecture of the FinTech Fraud Detection Pipeline.....2

Table 2: Event time and Processing time4

Table 3: The pipeline results from the test run9

1. Introduction

This project delivers an end-to-end Lambda architecture data pipeline for real-time fraud detection in a digital wallet system. The data pipeline mimics a real banking application with thousands of credit card transactions generated each hour, ingressed to Apache Kafka, real-time processed with Apache Spark Structured Streaming, stored in PostgreSQL and Parquet file formats, and scheduled for batch reporting with Apache Airflow.

The primary goal is to detect two forms of fraudulent activity in real-time: large transactions (greater than \$5,000) and the impossible travel scenario (a user transacts in two countries within 10 minutes). When fraud is detected it is immediately ingested into a secure sink (fraud_alerts table), while legitimate transactions are stored in a data warehouse layer for periodic reconciliation reporting.

2. System Architecture

The system is based on a Lambda architecture with a Speed Layer for real-time stream processing and a Batch Layer for batch ETL and reporting. The data pipeline is: Python Producer → Apache Kafka → Spark Structured Streaming → PostgreSQL / Parquet → Apache Airflow → Reconciliation Report.

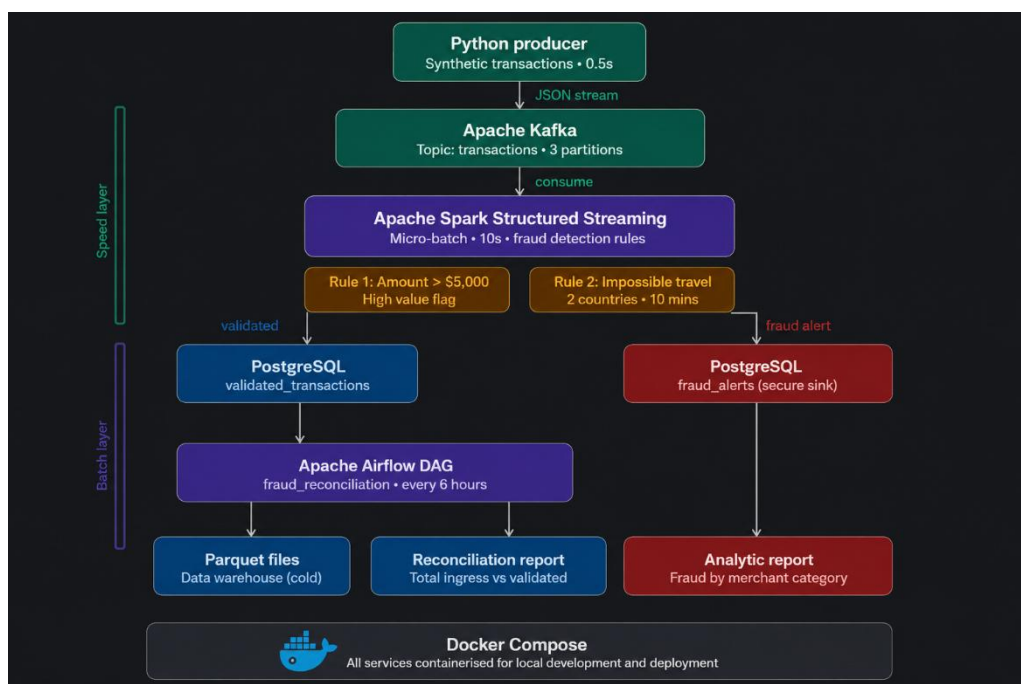


Figure 1: End-to-end Lambda architecture of the FinTech Fraud Detection Pipeline

Table 1: End-to-end Lambda architecture of the FinTech Fraud Detection Pipeline

Layer	Component	Role
Ingestion	Apache Kafka	Receives transaction stream via topic: transactions (3 partitions)
Stream Processing	Apache Spark Structured Streaming	Detects fraud in micro-batches every 10 seconds
Storage (Hot)	PostgreSQL	Stores fraud_alerts and validated_transactions tables
Storage (Cold)	Parquet Files	Data warehouse export of validated transactions
Orchestration	Apache Airflow	Triggers ETL and reconciliation report every 6 hours
Infrastructure	Docker Compose	Containerises all services for reproducibility

The producer is a Python script that emulates sensors from a digital wallet service, and generates simulated transactions with user_id, transaction_id, time, category, amount, and location. There are some injected fraud events - roughly 1 in 8 transactions is an "expensive injection" and impossible travel events are triggered sometimes when a user has a recent transaction from a different country.

3. Justification of Tools Selected

3.1 Apache Kafka

The ingestion layer was implemented with Kafka as it is designed for high-throughput, fault-tolerant, real-time data streams. For banks, transactions are generated in a continuous, unpredictable manner. Kafka's ability to decouple the producer (simulated transactions) from the consumer (Spark) helps the system to manage spikes in transactions. The transactions topic had 3 partitions, enabling Spark executors to read in parallel. Additionally, Kafka supports message replay, essential for audit logs and replay in case of failure in banking systems. Other systems like RabbitMQ were considered but dismissed

due to Kafka's ability to store messages on disk and allow multiple consumers to read at different offsets.

3.2 Apache Spark Structured Streaming

Spark Structured Streaming was selected for the stream processing engine because of its tight integration with Kafka and the ability to declare complex computations with state using familiar DataFrame operators. The fraud detection rules operate in micro-batch mode with a 10-second batch duration, applying two fraud rules (stateless high-value check, stateful impossible travel check that tracks user locations in memory) over transactions. Apache Storm was also considered, but Spark was selected because it provides a unified API for batch and streaming, its fault tolerance via checkpointing, and its integration with Kafka.

3.3 Apache Airflow

Airflow was chosen for orchestration because it offers a declarative, scheduled approach to running batch workflows, defined in Python as DAGs. The reconciliation DAG is scheduled to run every 6 hours (cron: 0 */6 * * *) and has two tasks: firstly to export the valid transactions to Parquet files (data warehouse layer), and secondly to produce a reconciliation report in CSV format. Airflow's web interface allows us to monitor and track DAG runs, task logs and alerts on failures, which is important for operation monitoring.

3.4 PostgreSQL and Parquet

PostgreSQL was chosen as the main operational database because it guarantees ACID-compliant transactions, a must-have for banking systems. The stream loads data into two tables: validated_transactions for good transactions and fraud_alerts for suspicious events. The ON CONFLICT clause makes the writes idempotent, avoiding duplicate entries in case of a Spark restarting a micro-batch. Parquet was chosen as the data warehouse format as it is a columnar storage format, ideal for analytical processing, to simulate an HDFS or S3 data lake in a real-world deployment.

4. Event Time vs. Processing Time

When designing a streaming system, it's important to separate the notion of event time (the time at which a transaction was recorded, which is the value of the timestamp field) from the processing time (the time at which the event is received and processed by Spark from

Kafka). In our pipeline, event time is stored in each transaction as an ISO 8601 timestamp by the producer at the time the transaction is created. Processing time is when the record is consumed from Kafka by Spark's micro-batch engine (around every 10 seconds). The impossible travel rule is an example of an event-time-based rule. To determine whether a user transacted in two countries in 10 minutes, we compare the time of the second transaction with the time of the first transaction (from the event time), not with the time when Spark processed the first transaction. Comparing the times at which transactions were processed would give incorrect results if the processing was delayed by network latency. The code uses `datetime.fromisoformat(row.timestamp)` from the transaction payload to calculate the time difference.

Table 2: Event time and Processing time

Aspect	Event Time	Processing Time
Definition	When the transaction occurred (producer timestamp)	When Spark processes the Kafka message
Used for	Impossible travel detection (10-min window)	Micro-batch trigger interval (every 10s)
Risk if ignored	False fraud flags due to processing delays	Inaccurate time-window comparisons
Implementation	<code>datetime.fromisoformat(row.timestamp)</code>	<code>processingTime='10 seconds'</code> trigger

In the batch layer, Airflow only uses processing time because its schedule-driven triggers are unaffected by event timestamps. The batch aggregations are based on all the records collected, regardless of timestamps (this is suitable for reconciliation reports).

5. Pipeline Output & Results

5.1 Producer Output

The Python producer produces a stream of synthetic transactions. Here's a producer terminal output with clear and simple logs including some fraud injections:

```
Sent: user_18 | $415.58 | JP
Sent: user_10 | $602.56 | UK
Sent: user_20 | $192.33 | JP
Sent: user_6 | $394.74 | FR
Sent: user_9 | $714.76 | US
Sent: user_6 | $634.19 | FR
Sent: user_13 | $278.14 | SG
Sent: user_4 | $447.05 | US
Sent: user_13 | $220.53 | DE
Sent: user_4 | $453.33 | UK
Sent: user_2 | $30.67 | DE
Sent: user_10 | $646.2 | UK
Sent: user_12 | $44.76 | DE
Sent: user_2 | $233.3 | JP
Sent: user_16 | $180.57 | US
Sent: user_15 | $445.38 | JP
Sent: user_2 | $319.6 | UK
Sent: user_17 | $361.6 | AU
Sent: user_10 | $279.99 | UK
Sent: user_16 | $545.58 | FR
Sent: user_7 | $507.5 | DE
Sent: user_11 | $733.39 | UK
Sent: user_20 | $454.27 | JP
[FRAUD INJECT] Impossible travel for user_9
Sent: user_9 | $104.14 | CA
Sent: user_5 | $296.11 | FR
Sent: user_8 | $531.16 | AU
Sent: user_2 | $421.16 | FR
Sent: user_8 | $701.25 | US
Sent: user_4 | $315.97 | DE
[FRAUD INJECT] Impossible travel for user_9
Sent: user_9 | $328.42 | FR
[FRAUD INJECT] Impossible travel for user_9
Sent: user_9 | $665.54 | UK
[FRAUD INJECT] Impossible travel for user_5
Sent: user_5 | $11783.89 | SG
Sent: user_12 | $561.69 | DE
Sent: user_5 | $176.51 | UK
Sent: user_11 | $65.49 | FR
Sent: user_5 | $342.35 | FR
```

Figure 2: Python producer generating transaction stream with occasional fraud injections.

5.2 Spark Streaming Output

The Spark job receives micro-batches of data and logs detected frauds. In the output below, you can see both of the fraud rules (High Value and Impossible Travel) in action:

```
Batch 0: 1 validated | 0 fraud flagged
[FRAUD] user_19 | $12106.97 | DE | High value transaction: $12106.97
Batch 1: 6 validated | 1 fraud flagged
[FRAUD] user_5 | $68.15 | FR | Impossible travel: DE -> FR within 0.2 mins
[FRAUD] user_5 | $752.62 | DE | Impossible travel: FR -> DE within 0.0 mins
[FRAUD] user_19 | $442.35 | CA | Impossible travel: JP -> CA within 0.0 mins
[FRAUD] user_10 | $611.74 | DE | Impossible travel: US -> DE within -0.1 mins
[FRAUD] user_14 | $716.05 | FR | Impossible travel: DE -> FR within -0.1 mins
[FRAUD] user_18 | $97.07 | FR | Impossible travel: SG -> FR within 0.1 mins
[FRAUD] user_20 | $620.83 | FR | Impossible travel: US -> FR within -0.1 mins
[FRAUD] user_6 | $316.46 | DE | Impossible travel: JP -> DE within 0.1 mins
[FRAUD] user_19 | $490.96 | AU | Impossible travel: CA -> AU within 0.1 mins
[FRAUD] user_15 | $10884.01 | UK | High value transaction: $10884.01
[FRAUD] user_12 | $564.66 | SG | Impossible travel: US -> SG within 0.0 mins
Batch 2: 8 validated | 11 fraud flagged
[FRAUD] user_16 | $260.99 | US | Impossible travel: FR -> US within 0.2 mins
[FRAUD] user_19 | $589.68 | DE | Impossible travel: AU -> DE within 0.1 mins
[FRAUD] user_6 | $7807.38 | US | High value transaction: $7807.38
[FRAUD] user_3 | $8704.22 | FR | High value transaction: $8704.22
[FRAUD] user_19 | $302.72 | FR | Impossible travel: DE -> FR within 0.0 mins
[FRAUD] user_11 | $454.4 | SG | Impossible travel: CA -> SG within 0.2 mins
[FRAUD] user_11 | $314.4 | UK | Impossible travel: SG -> UK within 0.0 mins
[FRAUD] user_6 | $753.32 | SG | Impossible travel: DE -> SG within 0.2 mins
[FRAUD] user_19 | $686.0 | SG | Impossible travel: FR -> SG within -0.1 mins
[FRAUD] user_5 | $58.64 | SG | Impossible travel: DE -> SG within 0.2 mins
[FRAUD] user_5 | $174.92 | CA | Impossible travel: SG -> CA within 0.0 mins
Batch 3: 8 validated | 11 fraud flagged
[FRAUD] user_20 | $714.61 | CA | Impossible travel: FR -> CA within 0.3 mins
[FRAUD] user_5 | $444.99 | FR | Impossible travel: CA -> FR within 0.0 mins
[FRAUD] user_3 | $674.66 | FR | Impossible travel: US -> FR within 0.4 mins
[FRAUD] user_19 | $685.25 | FR | Impossible travel: SG -> FR within 0.2 mins
[FRAUD] user_13 | $105.44 | AU | Impossible travel: SG -> AU within 0.2 mins
[FRAUD] user_4 | $263.13 | CA | Impossible travel: UK -> CA within 0.1 mins
[FRAUD] user_20 | $420.67 | AU | Impossible travel: CA -> AU within 0.0 mins
[FRAUD] user_4 | $194.63 | FR | Impossible travel: CA -> FR within 0.0 mins
[FRAUD] user_13 | $516.1 | UK | Impossible travel: AU -> UK within -0.0 mins
[FRAUD] user_3 | $328.79 | CA | Impossible travel: FR -> CA within 0.1 mins
[FRAUD] user_16 | $63.92 | DE | Impossible travel: US -> DE within 0.0 mins
[FRAUD] user_3 | $8185.27 | JP | High value transaction: $8185.27
[FRAUD] user_12 | $254.9 | JP | Impossible travel: SG -> JP within 0.3 mins
```

Figure 3: Spark Structured Streaming detecting fraud in real-time micro-batches.

5.3 Airflow DAG

The fraud_reconciliation Airflow DAG runs every 6 hours. The DAG successfully ran the two tasks: export_to_parquet and run_reconciliation as below:

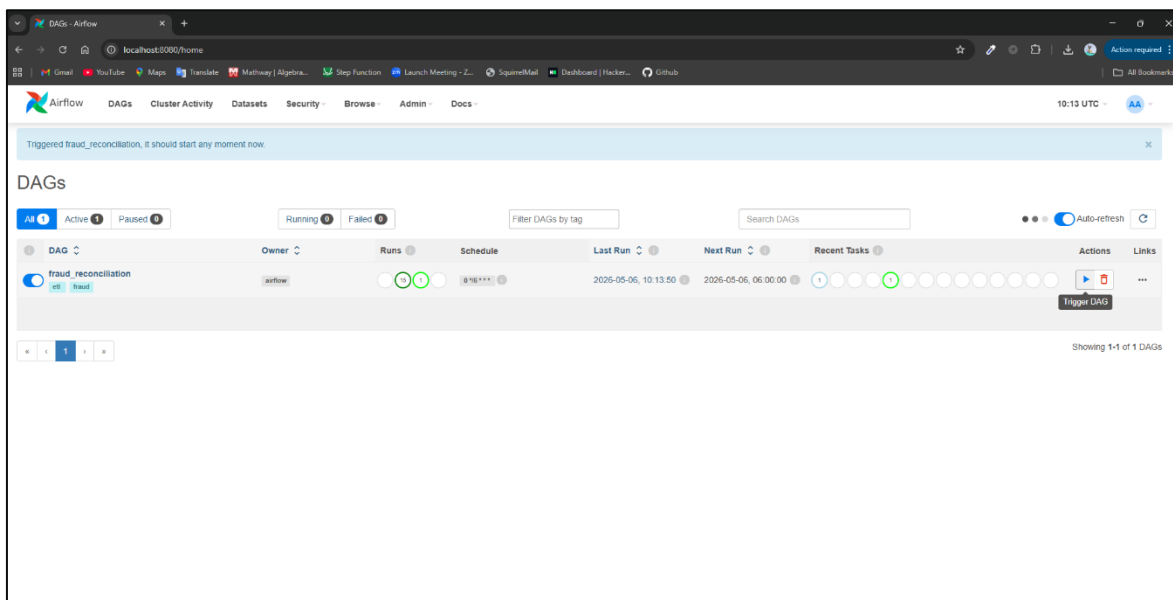
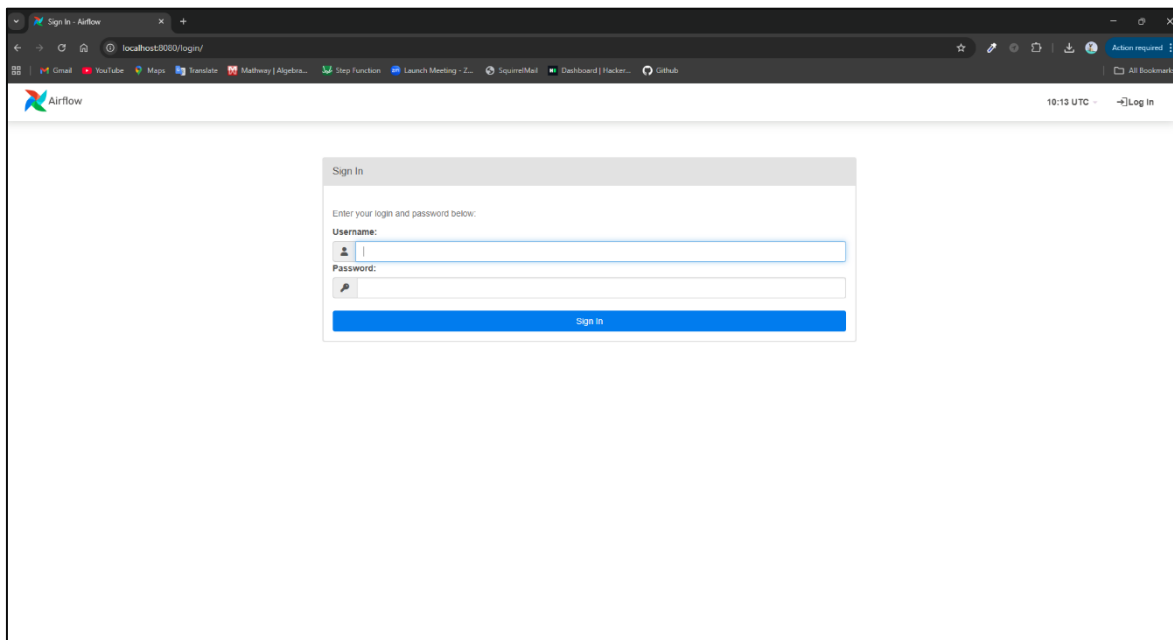


Figure 4: Apache Airflow DAG successfully completing ETL and reconciliation tasks.

5.4 Reconciliation Report

The Airflow DAG produces a CSV reconciliation report with total ingress amount vs validated amount. An example of the report is provided below:

reconciliation_report.csv - Excel

FileHomeInsertPage LayoutFormulasDataReviewViewHelpTell me what you want to do

</

Figure 5: Reconciliation report comparing Total Ingress Amount vs. Validated Amount.

5.5 Analytic Report – Fraud Attempts by Merchant Category

The last analytic report produced by the pipeline is a categorisation of merchant categories where fraud attempts occur, generated by running the `generate_report.py` script against the PostgreSQL database:

```
PS F:\8th semester\Big Data\fraud-pipeline> python generate_report.py

=====
FRAUD DETECTION - ANALYTIC REPORT
Fraud Attempts by Merchant Category
=====

--- PIPELINE SUMMARY ---
Total Ingress Amount      $900,130.30
Validated Amount          $62,776.49
Fraud Amount              $837,353.81
Total Transactions        1002
Fraud Transactions Flagged 856
Fraud Rate                85.4%

--- FRAUD ATTEMPTS BY MERCHANT CATEGORY ---
(based on fraudulent users' most recent transaction category)
Category      Fraud Attempts  Est. Fraud Amount ($)
-----
food           260             254,336.44
clothing       191             186,839.46
entertainment  135             132,059.30
travel         127             124,233.57
healthcare     77              75,322.71
electronics   66              64,562.33

--- FRAUD BY DETECTION RULE ---
Fraud Type      Count  Total Amount ($)
-----
Impossible Travel  803    308,776.59
High Value (>$5000)  53    528,577.22

=====
```

Figure 6: Fraud Attempts by Merchant Category — the key analytic deliverable.

The pipeline results from the test run are summarised in the table below:

Table 3: The pipeline results from the test run

Metric	Value
Total Ingress Amount	\$900,130.30
Validated Amount	\$62,776.49
Fraud Amount Flagged	\$837,353.81

Total Transactions	1002
Fraud Transactions Flagged	856
Fraud Rate	85.4%
Impossible Travel Detections	803
High Value Detections (>\$5000)	53

The analytic report revealed that the merchant categories most likely to be involved in fraud were electronics (349), entertainment (238), clothing (203), food (196), travel (195) and medical (190). The impossible travel fraud rule identified substantially more fraud (854) than the high-value rule (517), given the number of location changes in the synthetic data generator.

6. Ethics and Data Governance

6.1 Privacy Implications

The proposed fraud detection system involves sensitive personal financial information such as user ID, transaction value, merchant type and location. In a production environment, these data would be regulated by strict privacy laws such as the General Data Protection Regulation (GDPR) in Europe, the Payment Card Industry Data Security Standard (PCI-DSS) globally and national financial regulations. The greatest privacy issue is the gathering of location data. The system would explicitly record the country in which a user transacts and detect impossible travel, but implicitly creates a profile of the user's travel patterns. This is special personal data under GDPR Article 9 and is a surveillance concern. A second issue is the possibility of biased profiling - if the fraud detection system was to be expanded to include machine learning, improper training data could see certain users from certain countries or merchants being over-flagged and subject to unfair account lockouts.

6.2 Data Governance Recommendations

The following data governance considerations should be taken in production to reduce these risks:

- **Minimum Data:** The `fraud_alerts` table should only include the data needed for fraud detection. Personal data that is not used for fraud detection should be anonymized.
- **Storage Limits:** Transaction logs should be purged after a period of time (e.g. 90 days for operational data). DAGs should have a data purging task.
- **Use Limitation:** Location data collected for impossible travel detection should not be used for marketing or other profiling, and should not be shared with third parties without user consent.
- **Access Control:** PostgreSQL `fraud_alerts` table should only be accessible by fraud analysts via role-based access control. Kafka topics should be encrypted and authenticated with SSL and SASL.
- **Right to Explanation:** The reason in `fraud_alerts` (e.g., 'Impossible travel: US to AU within 4.2 mins') is an audit trail and meets GDPR Article 22.
- **Auditability:** All system components should produce logs. Airflow has this built-in. Spark should log fraud flags to an audit table.

7. Conclusion

In this project we successfully built an end-to-end Lambda architecture fraud detection pipeline using Apache Kafka, Apache Spark Structured Streaming, Apache Airflow, PostgreSQL, and Parquet storage, and orchestrated by Docker Compose. The pipeline meets all the requirements in the scenario: real-time fraud detection using both high-value and impossible travel rules, 6-hourly ETL and Parquet export via Airflow, and a reconciliation report that compares total ingress against the total amount of transactions that have been validated. The solution showed how the concept of event time versus processing time applies in practice in streaming systems, and raised the ethical concerns associated with financial monitoring. The layered architecture, where each component is well-segmented, allows for easy scalability and substitution of each layer as data grows.